

Стандартна библиотека на C++ - STL (Standard Template Library)

Какво е STL?

Какво представлява темплейт - просто начин да се абстрахираме от типа данни, с които работи дадена структура данни или алгоритъм.

STL представлява *библиотека от алгоритми и функции, реализирана на базата на темплейти*, за да може да работи с произволни типове данни.

Какво съдържа

Основни структури данни:

- динамичен масив (`vector`);
- опашка (`queue` и `deque`);
- свързан списък (`list`),
- стек (`stack`),
- асоциативен масив (`map` и `multimap`),
- множество (`set` и `multiset`),
- приоритетна опашка (`priority_queue` или `heap`), и
- (не точно в STL, но все пак) стринг (`string`).

Основни алгоритми:

- сортиране (`sort()`);
- обръщане (`reverse()`);
- размяна (`swap()`);
- минимум (`min()`);
- максимум (`max()`);
- премахване на дубликати (`unique()`);
- произволно разбъркване (`random_shuffle()`);
- следваща пермутация (`next_permutation()`);
- намиране на n -ти елемент (`nth_element()`).

Namespace

За да ползвате всички неща в STL, е нужно да включите пространството "std".

Това става с реда `using namespace std;` след `#include`-ите в началото на програмата.

Контейнери

За "структури данни" в STL се ползва името *контейнери*. Това име идва поради специфичността на структурите данни, които са реализирани в STL - това са само такива структури данни, които съдържат елементи (подредени по някакъв начин) и дават достъп до тях. Те рядко, ако изобщо, предоставят методи, които да връщат някаква по-специфична информация, както например тяхна сума, работа върху подинтервали или други подобни.

Ще разгледаме най-често ползваните от тях. Като цяло интерфейсът им е доста сходен (голяма част от методите се повтарят), в следствие на което нещото, което *реално* трябва да запомните е *разликите* между структурите данни.

pair

```
#include <utility>
```

Съхранява двойка обекти от произволен тип. Това става чрез *два* темплейта, K и T, вместо стандартния един (само T). Така примерно можете да имате двойки от тип `pair<string, int>`,

които съхраняват името на човек и неговите години. Често се ползва, когато искате да върнете две стойности, вместо само една, или искате да сортирате двойки елементи (примерно точки в равнината). Ако забелязвате, това е по-бърза за писане алтернатива на това, да създадете структура с две стойности.

Малко скрито остава това, че за нея са предефинирани и операторите <, == и т.н. Те са направени да работят по следния начин: първо се сравняват първите елементи от двете двойки, а ако те са еднакви се сравняват вторите.

Типът се задава по следния начин: `pair <K, T>`.

За да създадете двойка, може да ползвате функцията `make_pair(K val1, T val2)`, която връща `pair<K, T>` тип, със стойности `val1` и `val2`.

Можете да достъпите първия елемент чрез `.first`, а втория - чрез `.second`.

Забележете, че това е *директен достъп* до данните - `first` и `second` *не са* методи - и затова няма скоби () след тях!

Тъй като е темплейтен тип, то няма проблем едната от стойностите да е друг `pair`. Например, ако искате бързо да направите структура, която съдържа координатите на двумерна точка с целочислени координати и някаква `double` стойност (примерно тегло на въпросната точка), можете да направите:

```
#include <stdio.h>
#include <utility>
using namespace std;
int numPoints;
const int MAX_POINTS = 1024;
pair <pair<int,int>,float>points[MAX_POINTS];
void initPoints() {
    numPoints = 0 ;
    points[numPoints++] = make_pair(make_pair(42,13),3.141592);
    points[numPoints++] = make_pair(make_pair(17,17),1.618034);
    points[numPoints++] = make_pair(make_pair(-2,11),2.718282);
    printf("First point value: %f\n",points[0].second);
    printf("First point coordinates: (%d, %d)\n",points[0].first.first,
points[0].first.second);
    printf("Second point value: %f\n",points[1].second);
    printf("Second point coordinates: (%d, %d)\n",points[1].first.first,
points[1].first.second);
    printf("Third point value: %f\n",points[2].second);
    printf("Third point coordinates: (%d, %d)\n",points[2].first.first,
points[2].first.second);
}
int main()
{
    initPoints();
}
```

vector

```
#include <vector>
```

STL-ска имплементация на динамичен масив – размерът му може да се променя по време на изпълнение на програмата.

Съвет: не ползвайте вектори за щяло и нещяло. Когато можете да ползвате обикновен масив - ползвайте обикновен масив. Понякога програма, която ползва вектори, може да е *няколко пъти*

по-бавна от аналогична програма със стандартни масиви. Често без STL бихме ползвали повече памет, но какво от това? Ако програмата се събира в ограниченията по памет, то няма проблем.

Създаване:

`vector<тип> име;`

Например: `vector<int> MyVector;`

Възможни действия:

- `MyVector[i]` - връща елемента на позиция *i*, все едно ползвате обикновен масив.
- `MyVector.at(i)` - друг начин за вземане на елемента на позиция *i*.
- `MyVector.front()` - връща първия елемент от вектора.
- `MyVector.back()` - връща последния елемент от вектора.
- `MyVector.push_back(val)` - добавя нов елемент със стойност *val* в края на вектора.
- `MyVector.pop_back()` - премахва последния елемент на вектора.
- `MyVector.insert(pos, val)` - добавя елемента *val* на позиция *pos*, измествайки елементите от *pos* нататък надясно.
- `MyVector.resize(len)` - увеличава или намалява размера на вектора до *len*.
- `MyVector.resize(len, val)` - увеличава или намалява размера на вектора до *len*, като ако го увеличава, новите елементи са със стойност *val*.
- `MyVector.clear()` - изтрива всички елементи от вектора.
- `MyVector.erase(pos)` - изтрива елемента на позицията, сочена от *pos*, измествайки елементите от *pos* нататък наляво.
- `MyVector.erase(begin, end)` - изтрива всички елементи в интервала [*begin*, *end*), измествайки елементите от *end* нататък наляво.
- `MyVector.size()` - връща колко елемента има в масива.
- `MyVector.empty()` - връща `true` или `false` в зависимост дали `MyVector.size() == 0`.
- `MyVector.begin()` - връща итератор към началото на елементите.
- `MyVector.end()` - връща итератор към края на елементите.
- `MyVector.rbegin()` - връща обратен итератор към началото на елементите.
- `MyVector.rend()` - връща обратен итератор към края на елементите.
- `sort(MyVector.begin(), MyVector.end());` - сортиране

Итератори

Итераторите са специални променливи, които ни дават възможност да обхождаме `vector`.

string

```
#include <string>
```

Това е по-интересна употреба на `vector`, където типът е предварително зададен (`char`). С нея могат да се представят стрингове. Но като цяло има няколко особени предимства пред нормалния `char*`.

- Първо, размерът му може да бъде взет с константна сложност чрез метода `.size()`.
- Второ, не се нуждае от терминираща нула.
- Трето, позволява лесно разширяване или смаляване (както при вектор).
- Четвърто, копирането му е много по-лесно.
- Пето, предефинирани са няколко допълнителни оператора - например `operator <`, `operator +`, `operator +=`. С тях може да се сравняват и съединяват стрингове сравнително по-лесно, отколкото при `char*`.

Недостатъците са, че отново, както и при векторите, е малко по-бавен от нормален масив от `char`-ове. Има допълнителен метод `.c_str()`, който връща `const` указател към стринга, за да може да бъде печатан като обикновен C-style стринг. Това става по следния начин:

```
string myName = "Slim Shady";
```

```
printf(My name is %s.\n",myName.c_str());
```

Методите, които предоставя, са:

- `[int idx]` - връща символа на позиция *idx*, все едно ползвате обикновен масив.
- `.at(int idx)` - връща символа на позиция *idx*.
- `.find(char symbol)` - връща първата позиция (индексирано от нула), на която се среща символът *symbol*. Ако символът не се среща в стринга, вместо това бива върната константата `string::npos`, която е число, по-голямо или равно на дължината на стринга или -1.
- `.find(string str)` - връща първата позиция (индексирано от нула), на която се среща стрингът *str*. Ако аргументът не се среща в стринга, вместо това бива върната константата `string::npos`, която е число, по-голямо или равно на дължината на стринга или -1.
- `.substr(int start)` - връща друг стринг, започващ от *start*-ия елемент на стринга.
- `.substr(int start, int len)` - връща друг стринг, започващ от *start*-ия елемент на стринга с дължина *len*.
- `.c_str()` - връща константен указател към C-string (терминиран с 0 char масив), със същото съдържание като стринга.
- `.push_back(char val)` - добавя нов символ в края на стринга.
- `.append(string str)` - закача *str* накрая на стринга.
- `operator +=(string str)` - същото като `.append()`.
- `.insert(iterator pos, string str)` - добавя стринга *str* на позиция *pos*, измествайки символите от *pos* нататък надясно.
- `.resize(int len)` - увеличава или намалява размера на стринга до *len*.
- `.resize(int len, char val)` - увеличава или намалява размера на стринга до *len*, като ако го увеличава, новите елементи са със стойност *val*.
- `.clear()` - изтрива всички символи от стринга.
- `.erase(iterator pos)` - изтрива символа на позицията, сочена от *pos*, измествайки следващите символи наляво.
- `.erase(iterator begin, iterator end)` - изтрива подстринга в интервала `[begin, end)`, измествайки символите от *end* нататък наляво.
- `.size()` - връща колко елемента има в масива.
- `.length()` - същото като `.size()`.
- `.empty()` - връща `true` или `false` в зависимост дали `.size() == 0`.
- `.begin()` - връща итератор към началото на стринга.
- `.end()` - връща итератор към края на стринга.

deque

```
#include <queue>
```

Декът (*deque*) е опашка, в която можем да добавяме и премахваме елементи както в началото, така и в края. Обикновено, за да става това достатъчно бързо, е реализирана или чрез списък, или чрез циклична опашка (тоест в динамичен масив).

Методите, които предоставя, са:

- `[int idx]` - връща елемента на позиция *idx*.
- `.at(int idx)` - друг начин, за вземане на елемента на позиция *idx*.
- `.front()` - връща елемента в началото на опашката.
- `.back()` - връща елемента в края на опашката.
- `.push_front(T val)` - добавя нов елемент със стойност *val* в началото на опашката.
- `.push_back(T val)` - добавя нов елемент със стойност *val* в края на опашката.
- `.pop_front()` - премахва елемент от началото на опашката.
- `.pop_back()` - премахва елемент от края на опашката.

- `.resize(int len)` - увеличава или намалява размера на опашката до *len*. Ако се налага да се премахват елементи, това става от края на опашката.
- `.resize(int len, T val)` - увеличава или намалява размера на опашката до *len*, като ако го увеличава, новите елементи са със стойност *val*.
- `.clear()` - изтрива всички елементи от опашката.
- `.size()` - връща колко елемента има в опашката.
- `.empty()` - връща `true` или `false` в зависимост дали опашката е празна.
- `.begin()` - връща итератор към началото на елементите.
- `.end()` - връща итератор към края на елементите.

stack

```
#include <stack>
```

Поддържа стандартните операции за стек. Стандартната имплементация е реализирана на базата на deque.

Методите, които предоставя, са:

- `.top()` - връща елемента на върха на стека *без да го премахва*.
- `.push(T val)` - добавя нов елемент със стойност *val* на върха на стека.
- `.pop()` - премахва елемента, намиращ се на върха на стека.
- `.size()` - връща колко елемента има в стека.
- `.empty()` - връща `true` или `false` в зависимост дали стекът е празен или не.

queue

```
#include <queue>
```

Поддържа стандартните операции за опашка. Стандартната имплементация е реализиран на базата на deque.

Методите, които предоставя, са:

- `.front()` - връща елемента в началото на опашката *без да го премахва*.
- `.back()` - връща елемента в края на опашката *без да го премахва*.
- `.push(T val)` - добавя нов елемент със стойност *val* в края на опашката.
- `.pop()` - премахва елемента, намиращ се в началото на опашката.
- `.size()` - връща колко елемента има в опашката.
- `.empty()` - връща `true` или `false` в зависимост дали опашката е празна или не.

set

```
#include <set>
```

Тази STL-ска имплементация на set пази множество от елементи. Той, обаче, ползва различна структура на елементите ([балансирано двоично дърво](#)), в следствие на което постига по-добри сложности за добавяне, изтриване на елемент, като запазва същата за търсене. За сметка на това не ни дава достъп до *k*-тия по големина елемент. Все пак тази структура данни си остава много удобна в редица задачи (а е относително сложна, ако искате да имплементирате сами). Всъщност това (заедно с map, който е базиран на същото дърво) е може би най-сложното нещо, имплементирано в STL.

Методите, които предоставя, са:

- `.find(T val)` - връща итератор към елемента със стойност *val*, или итератора `.end()`, ако елементът не се намира в множеството.
- `.count(T val)` - намира колко на брой елемента със стойност *val* има в множеството. Тъй като това е *множество*, а не *мултимножество*, този брой е винаги или 0, или 1.
- `.lower_bound(T val)` - връща итератор към първия елемент от множеството, който е *по-голям или равен* на *val*.

- `.upper_bound(T val)` - връща итератор към първия елемент от множеството, който е *по-голям* от `val`.
- `.insert(T val)` - добавя елемент със стойност `val`, ако такъв вече не съществува.
- `.clear()` - изтрива всички елементи от множеството.
- `.erase(T val)` - изтрива елемента със стойност `val`, ако такъв съществува.
- `.erase(iterator it)` - изтрива елемента, сочен от итератора `it`.
- `.erase(iterator begin, iterator end)` - изтрива всички елементи, в интервал, зададен от два итератора. Забележете, че интервалът е полу-затворен: `[it1, it2)`.
- `.size()` - връща колко елемента има в множеството.
- `.empty()` - връща `true` или `false` в зависимост дали `.size() == 0`.
- `.begin()` - връща итератор към началото на елементите.
- `.end()` - връща итератор към края на елементите.

Тъй като `set` изисква наредба на елементите, за да работи с дефинирани от вас структури или класове, те трябва да имат предефинирани `operator <` (за вкарване) и `operator ==` (за търсене). Ако искате да *позволите* съществуването на дубликатни стойности, можете да ползвате почти аналогичния `multiset`.

map

```
#include <map>
```

Това е може би най-полезната структура данни в STL след `vector`. Тя дава *асоциативен масив*, тоест масив от двойки `<key, value>`, като по даден ключ (*key*) ви връща стойност (*value*). Например, в тази структура можете да запишете имената и ЕГН-тата на всички хора в България. Имената (от тип `string`) ще се ползват за ключ, а ЕГН-тата - за стойност (като `string` или `long long`). Така по дадено име структурата данни ще връща ЕГН-то, отговарящо за съответния човек. Нещо повече, можете да направите обратното, по ЕГН да ви връща името на човека! Така ключът ви ще е `long long` или `string` (в зависимост как предпочитате да пазите ЕГН-то), а стойността - `string`.

Как става това? Отново с два темплейта, както беше при `pair` - един за ключа и един за стойността. Всъщност `map` е почти същото нещо, като `set`, чиито елементи са `pair<K, T>`. Типовете `K` и `T` може да са както различни, така и еднакви.

Ако забелязвате, тази структура данни *надгражда* нещата, които ни даваше `set` и донякъде обезмисля ползването му. Все пак е добре да ползвате `set`, когато ви трябва множество, и `map`, когато ви трябва асоциативен масив.

Методите, които предоставя, са:

- `[K key]` - връща *стойността*, съответстваща на ключа `key`. Ако такава стойност няма, то не е дефинирано какво точно се случва, затова *винаги*, когато не сте *абсолютно сигурни*, че елемент с даден ключ присъства в асоциативния масив, проверявайте с `.count(key)` или `.find(key) != .end()` дали това е така. Това е и стандартният начин да присвоявате стойности - например `q["Alexander"] = 42;` (където `q` е асоциативен масив от тип `<string, int>`) би присвоило стойност 42 на елемента с ключ "Alexander", или би създавало такъв, ако все още не съществува.
- `.find(T val)` - връща итератор към елемента с ключ `val`, или итератора `.end()`, ако няма елемент с такъв ключ.
- `.count(T val)` - намира колко на брой елемента с ключ `val` има в асоциативния масив. Тъй като не са разрешени повтарящи се ключове, този брой е винаги или 0, или 1.
- `.lower_bound(T val)` - връща итератор към първия елемент от асоциативния масив, чиито ключ е *по-голям или равен* на `val`.

- `.upper_bound(T val)` - връща итератор към първия елемент от асоциативния масив, който е *по-голям* от *val*.
- `.insert(pair<K key, T val>)` - добавя елемент с ключ *key* и стойност *val*, ако не съществува елемент с такъв ключ. Ако такъв елемент вече съществува, то стойността му бива променена на *val*.
- `.clear()` - изтрива всички елементи от асоциативния масив.
- `.erase(T val)` - изтрива елемента с ключ *val*, ако такъв съществува.
- `.erase(iterator it)` - изтрива елемента, сочен от итератора *it*.
- `.erase(iterator begin, iterator end)` - изтрива всички елементи, в интервал, зададен от два итератора. Забележете, че интервалът е полу-затворен: `[it1, it2)`.
- `.size()` - връща колко елемента има в множеството.
- `.empty()` - връща `true` или `false` в зависимост дали `.size() == 0`.
- `.begin()` - връща итератор към началото на елементите.
- `.end()` - връща итератор към края на елементите.

Тъй като `map` изисква наредба на *ключовете* (тоест *K*), за да работи с дефинирани от вас структури или класове, те трябва да имат предефинирани `operator <` (за вкарване) и `operator ==` (за търсене).

Това изискване, както при `set`, така и при `map` възниква поради факта, че елементите се държат в сортиран вид през цялото време. А за да сортираме елементи, то трябва те да са сравними. Забележете, че *стойността* (тоест *T*) може да е от произволен тип и *няма* изискване да е сравнима. Ако искате да *позволите* съществуването на дубликатни стойности, можете да ползвате почти аналогичния `multimap`.

Итериране на `set` и `map`

Да обходите елементите на `set` или `map` не е едно от най-лесните неща. Има сравнително лесен `workaround`, ако не целите скорост и имате памет. Прехвърляте елементите на `set`-а или `map`-а във вектор и обхождате вектора. Това става по следния начин:

```
void printElementsOfSet(set<int>s) {
    vector<int>v(s.begin(),s.end());
    for(int i =0;i <(int)v.size();i++)
        printf("%d\n",v[i]);
}

void printElementsOfMap(map<double,int>m) {
    vector<pair<double,int>>v(m.begin(),m.end());
    for(int i =0;i <(int)v.size();i++)
        printf("(%lf, %d)\n",v[i].first,v[i].second);
}
```

Алтернативно, можете да ползвате малко по-сложния, но директен начин за обхождане на `set` или `map`. Както ще забележите, неговия синтаксис не е много интуитивен:

```
void printElementsOfSet(set<int>s) {
    for(set<int>::iterator it =s.begin();it !=s.end();it++)
        printf("%d\n",*it);
}

void printElementsOfMap(map<double,int>m) {
    for(map<double,int>::iterator it =m.begin();it !=m.end();it++)
        printf("(%lf, %d)\n",it->first,it->second);
}
```

Алгоритми

```
#include <algorithm>
```

Всъщност освен няколкото по-сложни алгоритъма (като, например, сортиране), остатъка от STL-ските "алгоритми" са няколкоредови рутинни парчета код (като търсене на елемент, броене, сумиране и т.н.). Съветвам ви да погледнете пълния списък [ето тук](#) и да разгледате тези от тях, които смятате, че ще ви бъдат полезни. Тук ще покажем само някои от тях.

min

```
min(T a, T b)
```

Връща минимума на два елемента *от еднакъв тип T*.

Би върнало 42 за `min(42, 666)`, 13 за `min(13, 13)` и -2 за `min(1, -2)`.

Би дало грешка при компиляция за `min(42, 13.0)`.

max

```
max(T a, T b)
```

Връща максимума на два елемента *от еднакъв тип T*.

Би върнало 666 за `max(42, 666)`, 13 за `max(13, 13)` и 1 за `max(1, -2)`.

Би дало грешка при компиляция за `max(42, 13.0)`.

swap

```
swap(T a, T b)
```

Разменя местата на елементите *a* и *b*. Тази толкова проста функцийка е много удобна, тъй като ако искаме сами да разменим два елемента, ще ни трябва допълнителна променлива и (общо) три реда код. Чрез `swap(a, b)` много по-ясно си личи какво искаме да направим.

sort

```
sort(итератор_към_началото, итератор_към_края)
```

И тъй като изобщо не искам да ви занимавам с това какво е итератор, то можете да си го представяте, че е просто указател. Тази представа всъщност е много близо до истината, тъй като можете да сортирате обикновен масив с числа (или обекти с дефиниран `operator <`) с подаване на указател към началото на масива и указател към позицията *след* последния елемент от интервала, който искате да сортирате. Забележете, че двата указателя задават полу-отворен интервал [`begin`, `end`). С други думи, за да сортирате масив с обекти от тип `Student`, за който сме дефинирали `operator <`, правим следното:

```
struct Student {
    string name;
    int egn;
    bool operator<(const Student&r) const {
        // Сортираме учениците по име
        if(name !=r.name)
            return name <r.name;
        // Ако са с еднакво име, ги сортираме по възраст
        // (по-възрастните ще са по-напред в списъка)
        else
            return egn >r.egn;
    }
};

const int MAX_STUDENTS =1000;
int numStudents;
Student students[MAX];
```

```
int main(
void ) {
// Някакъв код, който инициализира масива
// ...
sort(students, students + numStudents);
// ..
// Някакъв код, който работи със сортирания масив
return 0;
}
```

Сега да видим как можем да сортираме някои от другите структури данни, които вече разгледахме. Всъщност сортирането има смисъл само за някои от тях - например би било нелогично да сортираме опашка или стек. Свързан списък не можем да сортираме, тъй като липсва условието да имаме произволен достъп до елементите. Сетът вече е сортиран (както и STL-ския такъв). Остана динамичен масив, т.е. `vector`.

В нашата имплементация имаме достъп до масива и броя елементи, та лесно можем да ползваме стандартното сортиране на масиви. В STL-ския вектор, обаче, не можем да направим това. За щастие, при него са дадени два метода, които ни връщат *точно* тези итератори - те са `.begin()` и `.end()`. Така за да сортираме вектор от елементи с тип `Student` (ползваме същата структура от предходния пример) правим:

```
vector<Student>students;
int main(void) {
// Някакъв код, който инициализира вектора
// ...
sort(students.begin(), students.end());
// ..
// Някакъв код, който работи със сортирания вектор
return 0;
}
```

reverse

`reverse(итератор_към_началото, итератор_към_края)`

Обръща интервал наобратно. Ако е съдържал числата (7, 4, 42, 13, 666), то след извикването му ще бъде (666, 13, 42, 4, 7). Много хора го ползват веднага след сортиране - ако искат елементите да са сортирани в обратен ред.

Забележете, че аргументите отново са итератори, указващи интервал. Всъщност доста от алгоритмите в STL имат подобна сигнатура.

random_shuffle

`random_shuffle(итератор_към_началото, итератор_към_края)`

Прави произволно разбъркване на елементите. Удобно, когато искате да направите тест за някоя задача или пишете рандомизиран алгоритъм.

next_permutation

`bool next_permutation(итератор_към_началото, итератор_към_края)`

Преподрежда елементите в интервала в следващата лексикографска пермутация. Например, след извикване върху (4, 1, 3, 2) ще получим (4, 2, 1, 3). Връща `false`, ако текущата пермутация е последната възможна.

Удобно е когато имаме задача, в която искаме да пробваме *всички* възможни пермутации (тоест някакъв тип изчерпващо решение). Забележете, че ако искате да направите *всички* пермутации, то

първоначално трябва поредицата да е *сортирана*, иначе бихте пропуснали всички пермутации преди началната.

Ето код, който намира и изпечатва всички възможни пермутации на числата {1, 3, 3, 7}.

```
#include <stdio.h>
#include <algorithm>
#include <vector>
using namespace std;

int main() {
    vector<int>perm;
    perm.push_back(1);
    perm.push_back(3);
    perm.push_back(3);
    perm.push_back(7);
    do
    { printf("(");
      for(int i =0;i <(int)perm.size();i++) {
          printf("%d",perm[i]);
          if(i +1>=(int)perm.size())
              printf(")\n");
          else
              printf(", ");
      }
      while (next_permutation(perm.begin(),perm.end()));
    } while (next_permutation(perm.begin(),perm.end()));
    return 0;
}
```

Резултатът от изпълнението е:

```
(1, 3, 3, 7)
(1, 3, 7, 3)
(1, 7, 3, 3)
(3, 1, 3, 7)
(3, 1, 7, 3)
(3, 3, 1, 7)
(3, 3, 7, 1)
(3, 7, 1, 3)
(3, 7, 3, 1)
(7, 1, 3, 3)
(7, 3, 1, 3)
(7, 3, 3, 1)
```

nth_element

void nth_element(итератор_към_началото, итератор_към_позицията, итератор_към_края)

Слага на дадена позиция елемента, който би бил на тази позиция ако целият масив е сортиран.

Полезно е най-често за намиране на медиана. Времето за работа в средния случай е $O(N)$.

Например, ако имаме числата {101, 13, 17, 1337, 666, 42, 10, 69} и вземем nth_element с аргумент $n = 4$, функцията би подредила числата по такъв начин, че на позиция 4 да е елементът 69: елементът на позиция 4 от сортирания вариант на масива (10, 13, 17, 42, 69, 101, 666, 1337).

Как изглежда ако го ползваме с масиви:

```
int arr[8] = {101,13,17,1337,666,42,10,69};
nth_element(arr,arr +4,arr +8);
printf("%d\n",arr[4]);
```

Или пък с вектори:

```
vector <int>v({101,13,17,1337,666,42,10,69});
nth_element(v.begin(),v.begin() +4,v.end());
printf("%d\n",v[4]);
```

operator < и operator ==

Както споменахме по-горе, за някои от структурите данни (`set`, `map`, `priority_queue`, ...) а също така за някои от алгоритмите (`sort()`, `next_permutation()`, `min()`, `max()`) е нужно елементите на множеството, с което работим, да са сравними - тоест при наличието на два от тях A и B да можем да кажем дали $A < B$, $A > B$ или $A == B$. Работейки със стандартни типове (`int`, `double`, ...), а както и с типове от STL (`string`, `set`, etc.) това е тривиално, тъй като операторите за сравнение са дефинирани. Ако искаме да ползваме наша собствена структура, обаче, трябва да извършим малко допълнителна работа.

За наше щастие, това става сравнително просто и не ни пречи да го пишем по време на състезание (изисква едва 2-3 реда). Вътре в структурата трябва да имаме метод със сигнатура `bool operator < (const T& other) const;`, където T е името на структурата или класа, който пишем, а *other* е просто име на променлива, което може да е всякакво. Например, ако правим клас за точки с целочислени координати в тримерното пространство, това би било:

```
struct Point3D
{
    int x,y,z;
    bool operator<(const Point3D&other) const{
        if(x !=other.x)    return x <other.x;
        if(y !=other.y)    return y <other.y;
        return z <other.z
    }
    bool operator==(const Point3D&other) const
    {
        return x ==other.x &&y ==other.y &&z ==other.z;
    }
};
```

В случая наредбата на точките би била първо по x , при равен x - по y , и при равен x и y по z .

Домашно: от Втори стъпки стр.225/ Упр.12.2; стр.226/Упр.12.3